IT24036

Title: Comparison of thread counting using static and non-static variable.

objective: By compiling this code:

- Understand the difference between static and non-static variables.
- Create and execute multiple threads.
- Observe shared and instance-level data in multithreaded program.
- Calculate the difference.
- Percentage the difference between two count.
- Analyze the thread-effect on result.
- Understand the race conditions and thread safety.

Code:

```
import java.util.concurrent.atomic.AtomicLong;
public class Rina_Thread implements Runnable {
static final AtomicLong safeStaticCounter = new
AtomicLong(0);
```

REDMI NOTE 12

```
static long unsafeStaticCounter = 0;
private long instanceCounter = 0;
private final long increments;
private final boolean threadSafe;
public Rina_Thread (long increments, boolean threadSafe){
        this.increments = increments;
        this.threadSafe = threadSafe;
}
@Override
public void run (){
   for (long i=0; i<increments; i++) {
        if (threadSafe){
            safeStaticCounter.incrementAndGet();
        }
        else{
            unsafeStaticcounter++;
        }
        instanceCounter++;
   }
}
```

REDMI NOTE 12

```
public static void main (String[] args) throws
InterruptedException {
    if (args.length != 3) {
        System.out.println ("Usage: java Rina_Thread
                <thread> <increments> <truefalse>");

        return;
    }
int thread;
long increments;
boolean threadSafe;
try {
    threads = Integer.parseInt (args[0]);
    increments = Long.parseLong (args[1]);
    threadSafe = Boolean.parseBoolean (args[2]);

}
Catch (NumberFormatException e) {
    System.out.println ("Error: threads and
        increments must be integers.");

    return;
}
```

REDMI NOTE 12

```
if (threads<1 || increments<0) {
    System.out.println("Error: threads must be
                        >=1 and increments >=0 ");

    return;
}

Safe static counter.set(0);
unsafe static counter=0;
Rina_Thread[] tasks = new Rina_Thread [threads];
Thread[] workers = new Thread [threads]
for (int i=0; i<threads ; i++) {
    tasks[i] = new Rina_Thread (increments,
                                threadSafe);
    workers[i] = new Thread (tasks[i], "Worker-"+
                             (i+1));

}
for (Thread t: workers) {
    t.start();
}
```

REDMI NOTE 12

```
for (Thread t : workers) t.join();
Long expected = (long) threads * increments;
Long staticCount = threadSafe ? safe
Static counter get() : unsafeStatic
counter;
long nonStaticTotal = 0;
for (Rina_Thread t : tasks) {
        nonStaticTotal += t.instanceCounter;
        long absDiff = Math.abs(staticCount -
                                nonStaticTotal);
}
```

IT24036

Compile and Run :-

```
javac Rina_Thread.java
java Rina_Thread 10 100000 True
Mode : Thread-Safe (AtomicLong)
Threads (N) : 10
Increments/Thread (M) : 100000
Expected count : 1000000
Static count : 1000000
Non-static Total : 1000000
Absolute difference : 0
percentage Difference : 0.0000%
```

REDMI NOTE 12